Reliable Data Transfer over UDP

Project guide: what every function does and how to use the four scripts

1. Overview

The project has four Python files that live in one folder. `rdt.py` is the protocol library; the other three import it.

File	Role
`rdt.py`	The reliable data transfer protocol. Turns unreliable UDP datagrams into a reliable, in-order, full-duplex byte stream (sequence numbers, CRC-32 integrity checks, ACKs, timeouts, retransmission, sliding window, paced sending).
`simulator.py`	A UDP relay placed between client and server that randomly drops, corrupts, duplicates, delays and reorders packets, so the protocol can be tested.
`server.py`	File transfer server: serves put / get / list requests from a directory, one client at a time.
`client.py`	File transfer client: `put`, `get` and `list` commands with progress display and SHA-256 verification.

How the pieces connect

```
client.py  <---- rdt.py ---->  [ simulator.py ]  <---- rdt.py ---->  server.py
 (port: any)                    (--listen-port 8000)                 (--port 9000)
                                forwards to --server-port 9000

Without the simulator, the client talks to the server's port directly.
With it, the client talks to the simulator's port instead.
```

2. Quick start

Open three terminals in the project folder. Requirements: Python 3.8 or newer, no third-party packages.

Step 1 - start the server

```
python3 server.py --port 9000 --root server_files
```

The server creates `server_files` if it does not exist and serves files from it.

Step 2 - start the simulator (optional)

```
python3 simulator.py --listen-port 8000 --server-port 9000 \
    --loss 0.1 --corrupt 0.05 --reorder 0.2 --duplicate 0.05
```

Step 3 - use the client

```
# upload   (use --port 8000 to go through the simulator, 9000 to go direct)
python3 client.py --port 8000 put myfile.pdf
python3 client.py --port 8000 put myfile.pdf --name renamed.pdf

# list files on the server
python3 client.py --port 8000 list

# download
python3 client.py --port 8000 get myfile.pdf
python3 client.py --port 8000 get myfile.pdf -o copy.pdf
```

Every transfer ends with *sha256 verified* on success. Press Ctrl-C in the simulator or server window to stop it; the simulator then prints its packet statistics.

Using a very low sending rate

The assignment asks for a low rate (for example under 500 bits per second). The sender pauses `--interval` seconds after every packet, so the payload rate is `mss * 8 / interval` bits per second. The default (`--interval 0.02`) is fast so that testing is quick. For the assignment's rate, use 20 seconds with the default 1024-byte packets (1024 * 8 / 20 = 410 bits/s) on **both** the client and the server, because each side paces its own sending:

```
python3 server.py --port 9000 --interval 20
python3 client.py --port 9000 --interval 20 put tiny.txt
```

At that rate even a 1 KB file takes about 20 seconds, so demonstrate with a very small file. ACKs, the handshake and the final FIN exchange are control traffic and are not paced.

3. How the protocol works

Packet format

Every UDP datagram is one packet, with a 19-byte header in network byte order followed by up to `mss` payload bytes.

Field	Size	Meaning
`type`	1 byte	SYN=1, SYNACK=2, DATA=3, ACK=4, FIN=5
`conn id`	4 bytes	Random id chosen by the client. Packets with a different id are ignored, so stale packets from an earlier connection cannot leak in.
`seq`	4 bytes	Sequence number of a DATA/FIN packet. In an ACK it is the *cumulative* acknowledgement: the next sequence number the receiver expects.
`ack`	4 bytes	Used only in ACKs: the sequence number being *selectively* acknowledged (the packet that triggered this ACK).
`length`	2 bytes	Payload length, used to detect truncation.
`crc32`	4 bytes	CRC-32 over the header (with this field zeroed) and the payload.

Reliability mechanisms

Problem	How the protocol handles it
Packet lost	The sender keeps a timer for every unacknowledged packet. When it expires, only that packet is retransmitted (selective repeat).
Packet corrupted	The receiver recomputes the CRC-32. A mismatch (or a wrong length) means the packet is silently discarded, which then looks exactly like a loss and is recovered by retransmission.
Packets reordered	The receiver stores out-of-order packets in a buffer keyed by sequence number and hands data to the application strictly in order.
Packet duplicated / retransmitted twice	A packet whose sequence number was already received is not delivered again, but it is re-acknowledged (the earlier ACK may have been lost).
ACK lost	ACKs carry a cumulative value as well as the selective one, so a later ACK also covers earlier lost ACKs. If nothing else arrives, the sender retransmits and the receiver re-ACKs.
Link under-used	Up to `--window` packets may be in flight unacknowledged (pipelining).
Peer disappears	After `--max-retries` retransmissions of one packet, or 30 s of silence, the connection fails and calls raise ConnectionError.

Connection lifecycle

Open: the client sends SYN (retransmitted until answered); the server replies SYNACK and starts the connection. If the SYNACK is lost, the client's repeated SYN is answered again. **Transfer:** both sides can send and receive at the same time. **Close:** each side sends a FIN, which is numbered and acknowledged like data, so it arrives after all earlier data. `close()` waits until its own FIN is acknowledged and the peer's FIN has arrived, then lingers three timeouts so a lost final ACK can still be repeated.

File transfer protocol (on top of the byte stream)

Requests and responses are one JSON object per line (ending in `\n`), followed by raw bytes where needed.

Command	Exchange
`PUT`	Client sends {cmd, name, size, sha256}. Server replies {status: ok} when ready. Client sends exactly size bytes. Server verifies the SHA-256 and replies {status: ok, size, sha256} or {status: error, message}.
`GET`	Client sends {cmd, name}. Server replies {status: ok, size, sha256} followed by size bytes, or {status: error, message}. The client verifies the SHA-256.
`LIST`	Client sends {cmd: LIST}. Server replies {status: ok, size} followed by size bytes of text, one `name<TAB>size` line per file.

4. rdt.py reference

Names beginning with an underscore are internal helpers; an application normally uses only `connect`, `Listener`, `Connection` and `Config`.

Constants

Name	Meaning
`SYN, SYNACK, DATA, ACK, FIN`	Packet type codes 1 to 5.
`TYPE_NAMES`	Maps a type code to its name; also used to reject packets with an unknown type.
`HEADER`	The `struct` format `!BIIIHI` of the 19-byte header.
`MAX_DATAGRAM`	65535, the receive buffer size for one datagram.
`RECV_WINDOW_FACTOR`	The receiver buffers out-of-order packets up to 4 windows ahead of the next expected one.
`POLL`	0.05 s. How often waiting threads wake up to re-check their conditions.
`Packet`	Named tuple `(ptype, conn_id, seq, ack, payload)` returned by decode_packet.

class Config

`Config(mss=1024, window=16, timeout=0.5, interval=0.02, max_retries=20, idle_timeout=30.0)`

A dataclass holding the tunable protocol parameters. Pass one to `connect()` or `Listener()`.

Field	Meaning
`mss`	Maximum payload bytes per packet.
`window`	Maximum number of unacknowledged packets in flight.
`timeout`	Retransmission timeout in seconds. Should be larger than the round-trip time, including any simulator delay.
`interval`	Pause in seconds after each transmission; sets the sending rate.
`max_retries`	Retransmissions of one packet before the connection is declared failed.
`idle_timeout`	Seconds without any valid packet before the peer is declared dead (at least 10 x interval is used).

Command-line helpers

`add_protocol_arguments(parser)`

Adds the shared options --mss, --window, --timeout, --interval and --max-retries to an `argparse.ArgumentParser`, with the Config defaults. Used by server.py and client.py so both accept the same tuning flags.

`config_from_args(args)`

Builds a Config from the namespace returned by `parser.parse_args()`.

Packet functions

`encode_packet(ptype, conn_id, seq=0, ack=0, payload=b'')`

Builds a datagram: packs the header with the CRC field set to zero, computes CRC-32 over header plus payload, then packs the header again with the real CRC and appends the payload. Returns `bytes`.

`decode_packet(raw)`

Parses a received datagram and verifies it. Returns a Packet, or `None` if the datagram is shorter than the header, has an unknown type, has a payload length that does not match the header, or fails the CRC check. Callers treat None as 'drop it'.

`describe_packet(raw)`

Returns a one-line description such as `DATA seq=12 len=1024` or `ACK cum=13 sel=12`. It does *not* verify the CRC, so it also works on corrupted packets. The simulator uses it for verbose logging.

class Connection

One reliable, in-order, full-duplex byte stream to a single peer. You do not construct it directly: `connect()` and `Listener.accept()` return one. Two background threads run the protocol: a **sender** thread (pacing and retransmission timers) and a **receiver** thread (verifies, orders and acknowledges packets). The public methods below are blocking and safe to call from your own thread. It supports `with`, which calls close() on exit. The `stats` attribute is a Counter with *sent*, *retransmitted*, *received*, *corrupt* and *duplicates*.

`Connection.peer (property)`

The remote address as `(ip, port)`.

`Connection.start()`

Sets the socket poll timeout and starts the sender and receiver threads. Called by connect() and accept(); you never call it.

`Connection.send(data)`

Splits `data` into mss-sized chunks and queues them for reliable delivery. Blocks while the send queue is full (2 x window chunks), which throttles the caller to the protocol's pace. Returns when the data is *queued*, not when it is acknowledged; close() waits for the acknowledgements. Raises ConnectionError if the connection failed or was already closed for writing.

`Connection.recv(max_bytes=65536)`

Returns up to max_bytes of in-order data, blocking until at least some is available. Returns `b''` once the peer has closed and everything was read (end of stream). Raises ConnectionError if the connection failed with no data left.

`Connection.recv_exact(count)`

Calls recv() repeatedly until exactly count bytes have been collected. Raises EOFError if the peer closes first.

`Connection.recv_line(max_length=65536)`

Blocks until a full newline-terminated line is available and returns it without the newline. Raises EOFError if the peer closes first, and ValueError if max_length bytes arrive without a newline. The file transfer headers are read with this.

`Connection.close()`

Graceful shutdown, and it never raises. It queues a FIN after all pending data, waits until every packet is acknowledged, waits for the peer's FIN (up to timeout x max_retries seconds), lingers 3 x timeout so a lost final ACK can be repeated, then stops the threads and, for client connections, closes the socket. Always call it, otherwise queued data may not be delivered.

`Connection.__enter__() / __exit__()`

Context-manager support: `with connect(addr) as conn:` closes the connection automatically.

Connection internals

Method	What it does
`_check_writable()`	Raises ConnectionError if the connection failed or close() was already called.
`_fail(reason)`	Records a failure reason, logs it, and wakes every thread waiting on the connection so they can raise.
`_next_packet()`	Decides what the sender thread transmits next (lock held). Expired retransmissions first (and gives up after max_retries); otherwise new queued data if the window has room, assigning the next sequence number.
`_send_loop()`	Sender thread: repeatedly takes the packet from _next_packet(), sends it, then sleeps *interval* seconds (the pacing).
`_recv_loop()`	Receiver thread: reads datagrams, ignores other senders and other connection ids, drops corrupt packets (counted in stats), dispatches DATA/FIN to _on_data and ACK to _on_ack, re-sends SYNACK if the peer repeats a SYN, and declares an idle timeout.
`_send_raw(raw)`	Sends a datagram immediately (used for ACKs and SYNACKs, which are not paced), ignoring socket errors.
`_on_data(pkt)`	Handles DATA/FIN: ignores packets too far ahead, counts duplicates, buffers new packets, delivers all consecutive packets starting at the expected sequence number to the receive buffer (a FIN marks end of stream), and sends an ACK with the cumulative and selective values.
`_on_ack(pkt)`	Handles an ACK: removes from the in-flight table every packet below the cumulative value plus the selectively acknowledged one, freeing window space.

Connection setup functions

`connect(address, config=None)`

Client side. Resolves the host, picks a random connection id and sends a SYN, retransmitting every timeout up to max_retries times until a valid SYNACK arrives. Returns an established Connection that owns its socket. Raises ConnectionError if nobody answers.

`address` is a `(host, port)` tuple.

`_resolve(address)`

Internal. Converts `(host, port)` to an IPv4 `(ip, port)` so incoming packets can be matched against the peer address.

`class Listener(host='0.0.0.0', port=9000, config=None)`

Server side. Binds a UDP socket. Serves one connection at a time: call accept(), finish with the connection (including close()), then call accept() again.

`Listener.accept(timeout=None)`

Waits for a valid SYN, answers with a SYNACK and returns a new Connection to that client. Anything that is not a valid SYN is ignored. With a timeout, raises TimeoutError when it expires; None waits forever.

`Listener.close()`

Closes the listening socket.

Using rdt.py in your own program

```
import rdt

# ---- server ----
listener = rdt.Listener("0.0.0.0", 9000)
conn = listener.accept()
line = conn.recv_line()            # b'hello'
conn.send(b"reply\n")
conn.close()

# ---- client ----
cfg = rdt.Config(interval=0.01, timeout=0.3)
with rdt.connect(("127.0.0.1", 9000), cfg) as conn:
    conn.send(b"hello\n")
    print(conn.recv_line())        # b'reply'
    print(dict(conn.stats))        # sent / retransmitted / corrupt ...
```

5. simulator.py reference

A UDP relay for testing. The first datagram from any address other than the server marks that address as the client; replies from the server are relayed back to it. Impairments are applied to every datagram in both directions, in this order: loss, corruption, duplication, then delay (with optional extra delay to cause reordering). Only one client at a time is supported.

`class Simulator(listen, server, loss=0.0, corrupt=0.0, reorder=0.0, duplicate=0.0, delay=0.0, jitter=0.0, reorder_delay=0.3, seed=None)`

Binds the relay socket on `listen` (host, port) and prepares to forward to `server` (host, port). Probabilities are between 0 and 1. A `seed` makes runs repeatable. `stats` counts received, dropped, corrupted, duplicated, reordered and forwarded packets.

`Simulator._corrupt_bytes(raw)`

Returns a copy of the datagram with 1 to 3 randomly chosen bytes XORed with a non-zero value, so the damage is always real.

`Simulator._handle(raw, src)`

Processes one received datagram: decides its direction, applies loss, corruption and duplication, computes its release time (delay + random jitter, plus extra random delay for 'reordered' packets so that later packets overtake it) and puts it in a time-ordered queue.

`Simulator._flush_due()`

Sends every queued datagram whose release time has arrived.

`Simulator.run()`

Main loop: waits for a datagram or for the next release time, handles it, flushes due packets. Runs until Ctrl-C, then logs the statistics and closes the socket.

`probability(text)`

argparse type function: converts text to a float and rejects values outside 0 to 1.

`main()`

Parses the command line, configures logging and runs a Simulator.

simulator.py options

Option	Meaning (default)
`--listen-host / --listen-port`	Where the relay listens (0.0.0.0 / 8000). Point clients here.
`--server-host / --server-port`	Where the real server is (127.0.0.1 / 9000).
`--loss P`	Probability a packet is dropped (0).
`--corrupt P`	Probability a packet has random bytes altered (0).
`--reorder P`	Probability a packet gets extra delay so later packets overtake it (0).
`--duplicate P`	Probability a packet is delivered twice (0).
`--delay S`	Fixed one-way delay in seconds (0). Keep the protocol --timeout above the round trip, about 2 x delay.
`--jitter S`	Random extra delay of 0 to S seconds (0).
`--reorder-delay S`	Maximum extra delay for reordered packets (0.3).
`--seed N`	Random seed for reproducible runs.
`-v`	Log every packet decision (DROPPED, CORRUPTED, ...).

6. server.py reference

`send_json(conn, **fields)`

Sends one JSON object followed by a newline over the connection. Used for all server responses.

`safe_path(root, name)`

Maps a client-supplied name to a path inside the served directory. Accepts only plain file names; rejects empty names, '.', '..', names containing a path separator, backslash or NUL byte (blocking path traversal such as ../secret). Raises ValueError otherwise.

`handle_put(conn, root, request)`

Upload. Validates the name and size, replies 'ready', receives exactly size bytes into `name.part` while computing a SHA-256, then compares with the client's hash. On a match the temp file is renamed into place and an ok response with the hash is sent; on a mismatch the temp file is deleted and an error is sent. A failed upload therefore never leaves a half-written file under the real name.

`handle_get(conn, root, request)`

Download. Raises FileNotFoundError if the file does not exist; otherwise computes its SHA-256, sends an ok response with size and hash, then streams the file in 64 KiB blocks.

`handle_list(conn, root, _request)`

Sends the number of bytes in the listing, then one `name TAB size` line per regular file (unfinished .part files are hidden).

`HANDLERS`

Dictionary mapping the command names PUT, GET and LIST to their handler functions.

`serve_connection(conn, root)`

Reads one JSON request line (at most 4096 bytes), runs the matching handler, and turns ValueError / FileNotFoundError / PermissionError into an error response. Connection problems are logged and do not crash the server.

`main()`

Parses options, creates the root directory, then loops forever: accept a client, serve it, close the connection. Ctrl-C shuts it down.

server.py options

Option	Meaning (default)
`--host / --port`	Address and UDP port to bind (0.0.0.0 / 9000).
`--root DIR`	Directory to serve (server_files).
`-v`	Verbose logging.
`--mss --window --timeout --interval --max-retries`	Protocol tuning, see section 4 (Config). Use the same values on client and server.

7. client.py reference

`send_json(conn, **fields)`

Sends one JSON object plus newline to the server (the request).

`read_response(conn)`

Reads one JSON response line. Returns the dictionary if its status is 'ok'; raises RuntimeError with the server's message otherwise.

`sha256_of(path)`

Returns the hex SHA-256 of a local file, read in 64 KiB blocks.

`progress(done, total, started)`

Prints a one-line percentage, byte count and KiB/s rate to stderr, overwriting the previous line.

`put_file(conn, path, remote_name)`

Upload. Hashes the file, sends the PUT request, waits for the server's 'ready', streams the file in 64 KiB blocks with progress, then reads the result and confirms the server stored the same hash.

`get_file(conn, remote_name, dest)`

Download. Sends the GET request, reads size and hash, writes the data to `dest.part` while hashing, and renames it to dest only if the hash matches (otherwise the download is discarded and an error is reported).

`list_files(conn)`

Sends the LIST request and prints the listing.

`main()`

Parses the command line, connects, runs the chosen command and always closes the connection. Prints 'error: ...' to stderr and exits with status 1 on failure, 0 on success. With -v it also prints the protocol statistics (sent, retransmitted, corrupt, ...).

client.py options

Command / option	Meaning (default)
`--host / --port`	Server or simulator address (127.0.0.1 / 9000). Place before the command.
`put FILE [-n NAME]`	Upload FILE; -n sets its name on the server (default: the file's own name).
`get NAME [-o PATH]`	Download NAME; -o sets the local destination (default: NAME in the current folder).
`list`	Print the files on the server.
`-v`	Verbose logging and protocol statistics at the end.
`--mss --window --timeout --interval --max-retries`	Protocol tuning, see section 4 (Config).

8. Testing the protocol

Suggested experiments for evaluating the protocol with the simulator (add `--interval 0.002 --timeout 0.3` to client and server to make them fast):

Goal	Simulator options
Loss only	--loss 0.2
Corruption only	--corrupt 0.2
Reordering only	--reorder 0.5 --reorder-delay 0.4
Duplication only	--duplicate 0.3
Everything at once	--loss 0.15 --corrupt 0.1 --reorder 0.25 --duplicate 0.1 --seed 7
Slow link	--delay 0.1 --jitter 0.05 (and client/server --timeout 0.6)

After each transfer, confirm the files are identical:

```
sha256sum original.bin server_files/original.bin downloaded.bin
```

Run the client with `-v` to see the *retransmitted* and *corrupt* counters, and the simulator with `-v` to see each packet decision. Comparing the retransmission count with the loss rate shows the cost of unreliability.

9. Troubleshooting

Symptom	Likely cause and fix
error: no response from ...	Server or simulator is not running, or the wrong port was given. The client tries the SYN max-retries times first.
error: peer unresponsive / idle timeout	The other side stopped, or loss is so high that a packet exceeded max-retries. Raise --max-retries or lower the loss.
Transfer is very slow	--interval is large. Lower it (the default is 0.02) unless you are deliberately demonstrating the low rate.
Many retransmissions even without loss	--timeout is shorter than the real round trip (for example a simulator --delay). Raise --timeout.
checksum mismatch	Data was damaged in a way the per-packet CRC missed. Report it; the transfer is discarded rather than stored.
ModuleNotFoundError: rdt	The scripts are not in the same folder, or you are running them from another folder. Run from the project folder.
Address already in use	Another server or simulator already uses that UDP port. Pick another port.

Known limits

One client is served at a time; the simulator also supports one client. Addresses are IPv4 only. The protocol has no congestion control, by design (the assignment asks for a fixed low rate), and a fixed rather than adaptive retransmission timeout. Neither connection setup nor ACKs are authenticated or encrypted.